

Employee & Department Service — REST + MCP Architecture

Item	Value
Status	Implemented (Phases 0–8). See §15 for where the build differs from the original draft
Date	2026-10-07
Stack	Java 21, Spring Boot 4.1.1, Spring AI 2.0.1 (MCP Server WebMVC starter, MCP Java SDK 2.0), Spring Data JPA, H2/PostgreSQL, Flyway, springdoc 3.1
Companion plan	../plans/2026-10-07-employee-department-mcp-implementation-plan.md

Version note: versions were pinned in Phase 0 from start.spring.io (Boot 4.1.1 + Spring AI 2.0.1, the current pair on 2026-10-07). The MCP layer depends on the `spring-ai-starter-mcp-server-webmvc` starter, the `@McpTool` / `@McpResource` / `@McpPrompt` annotations (`org.springframework.ai.mcp.annotation`) and the `spring.ai.mcp.server.*` properties.

1. Purpose and scope

The service manages **employees** and the **departments** they belong to. It exposes the same business capabilities through two interfaces:

1. A **REST API** (`/api/v1/**`) for conventional HTTP clients: web UIs, other services, Postman, curl.
2. An **MCP server** (`/mcp`) that publishes the same operations as **MCP tools**, **resources** and **prompts**, so any MCP-compatible client (Claude Code, Claude Desktop, VS Code / Copilot, Cursor, MCP Inspector, Spring AI MCP clients, custom agents) can discover and call them.

In scope

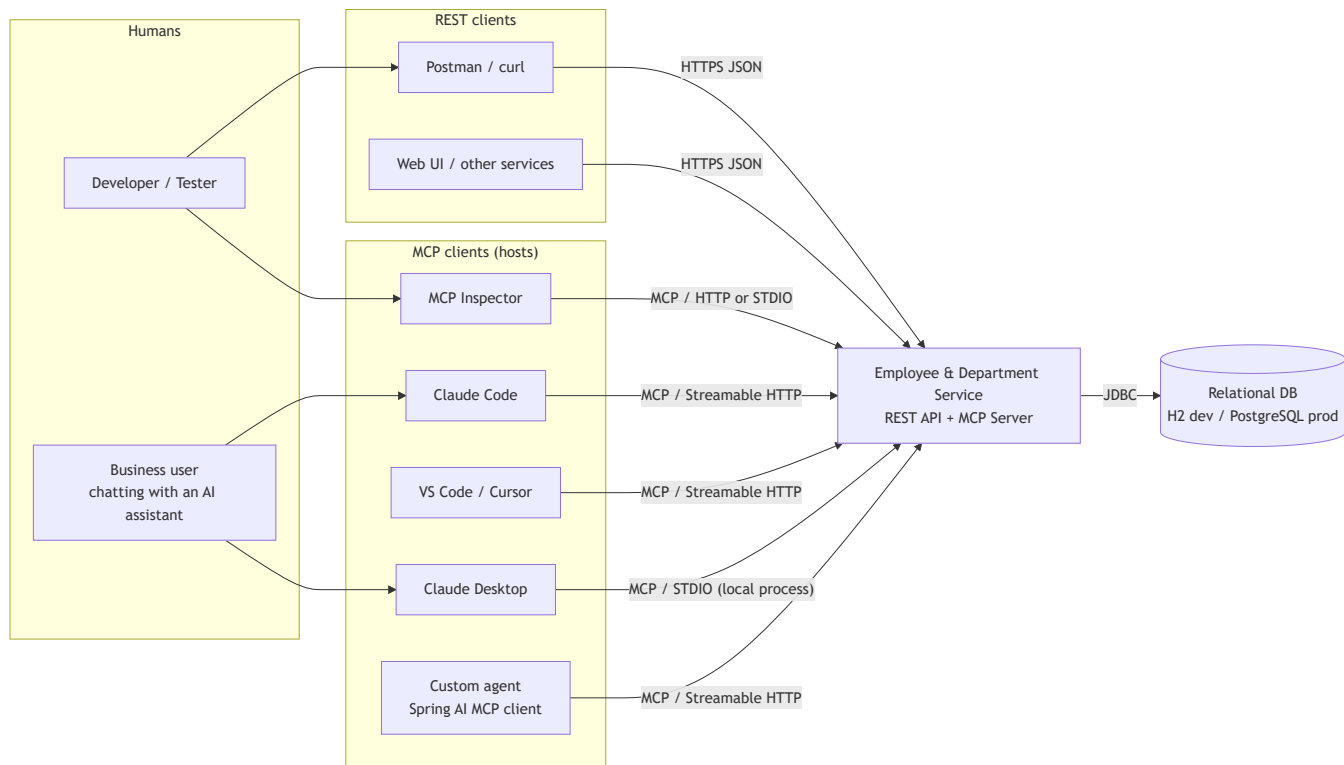
- CRUD for departments and employees
- Department ↔ employee relationship: list members, transfer an employee, department summary (headcount, payroll)
- Search and paging for employees
- MCP over **Streamable HTTP** (remote clients) and **STDIO** (local desktop clients)
- Validation, consistent error model, OpenAPI docs, tests, API-key protection, container image

Out of scope (for this sample)

- Multi-tenancy, full OAuth 2.1 authorization server, HR workflows (payroll, leave), UI

2. System context

Who talks to the system and how.

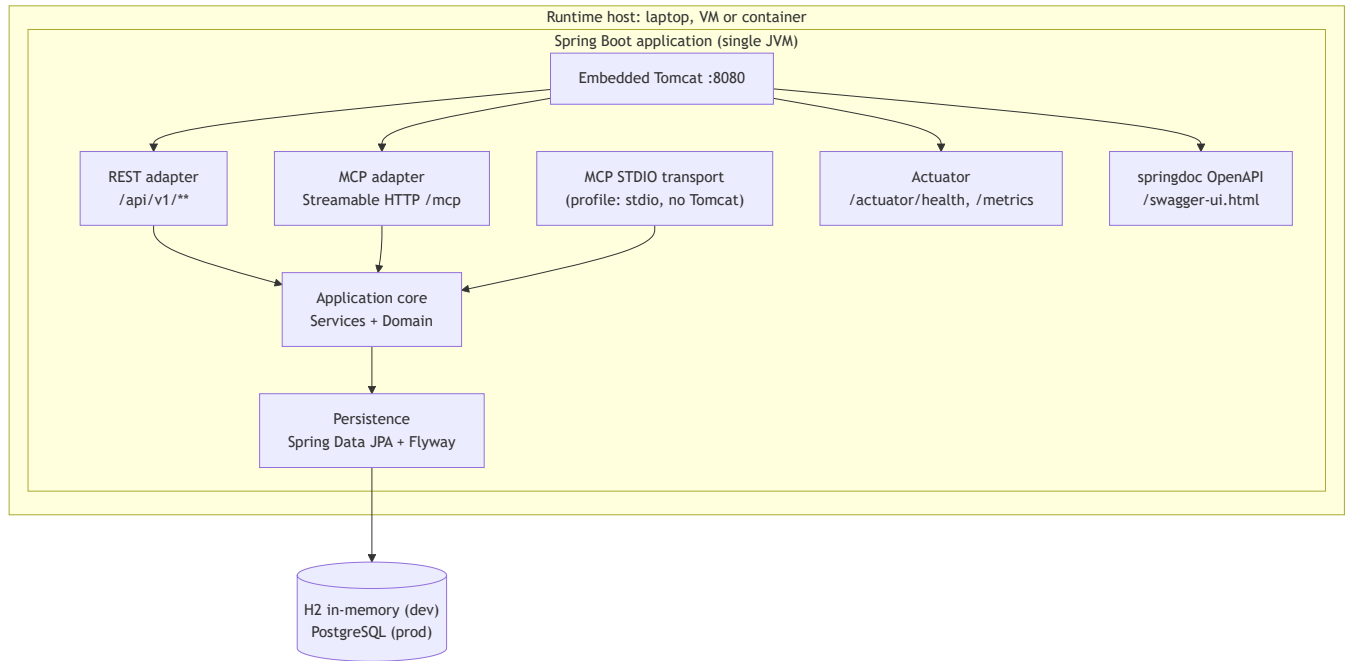


3. Key architectural decisions (ADR summary)

#	Decision	Chosen option	Alternatives considered	Rationale
ADR-01	How MCP tools reach business logic	In-process: MCP tools call the same Service layer as REST controllers	Separate MCP gateway app that calls the REST API over HTTP	One deployable, no double serialization, one validation path, transactional consistency. REST and MCP are two <i>adapters</i> over one core (hexagonal style). The gateway variant is described in §11 if the REST API must stay untouched or lives elsewhere.
ADR-02	MCP framework	Spring AI MCP Server Boot Starter (WebMVC) with @McpTool annotations	Generic @Tool + ToolCallbackProvider ; raw MCP Java SDK; WebFlux starter	Auto-configuration and annotation scanning, JSON schema generation from method signatures, and MCP tool annotations (readOnlyHint , destructiveHint) that hosts use when deciding to ask for confirmation. Fits a servlet (WebMVC) app that already serves REST.
ADR-03	Remote transport	Streamable HTTP at /mcp	Legacy HTTP+SSE (/sse + /mcp/message), stateless HTTP	Streamable HTTP is the current MCP spec transport; one endpoint; supported by Claude Code, VS Code, Inspector. SSE stays a config switch for old clients.
ADR-04	Local transport	STDIO via Spring profile studio	Only HTTP + mcp-remote bridge	Claude Desktop and similar hosts launch local servers as processes. The studio profile disables the web server and console logging so stdout stays a clean JSON-RPC channel.
ADR-05	Server type	SYNC	ASYNC (Reactor)	Blocking JPA calls; simpler code; adequate for the expected load.
ADR-06	Package layout	Package-by-feature (department , employee , mcp , common)	Package-by-layer	Keeps each aggregate cohesive; the mcp and web adapters stay thin.

#	Decision	Chosen option	Alternatives considered	Rationale
ADR-07	Schema management	Flyway migrations + seed data	<code>ddl-auto=update</code>	Repeatable, reviewable schema; same scripts for H2 and PostgreSQL.
ADR-08	Error contract	REST: RFC 9457 ProblemDetail . MCP: tool result with <code>isError=true</code> and a human-readable message	Custom error JSON	Standard formats; LLM clients can read the error text and recover.
ADR-09	Security (sample level)	API key (<code>X-API-KEY</code> or <code>Bearer</code>) on <code>/api/**</code> and <code>/mcp</code> , enforced when <code>APP_API_KEY</code> is set; localhost binding + Origin check for MCP always on	Full OAuth 2.1 (MCP authorization spec)	Easy for any client to configure. With no key the server is open but bound to 127.0.0.1 and logs a warning, so the local demo needs no setup. The OAuth 2.1 resource-server upgrade path is in §9.
ADR-10	DTO boundary	Java records for request/response; entities never leave the service layer	Expose entities	Avoids lazy-loading/recursion problems; gives clean tool JSON schemas.

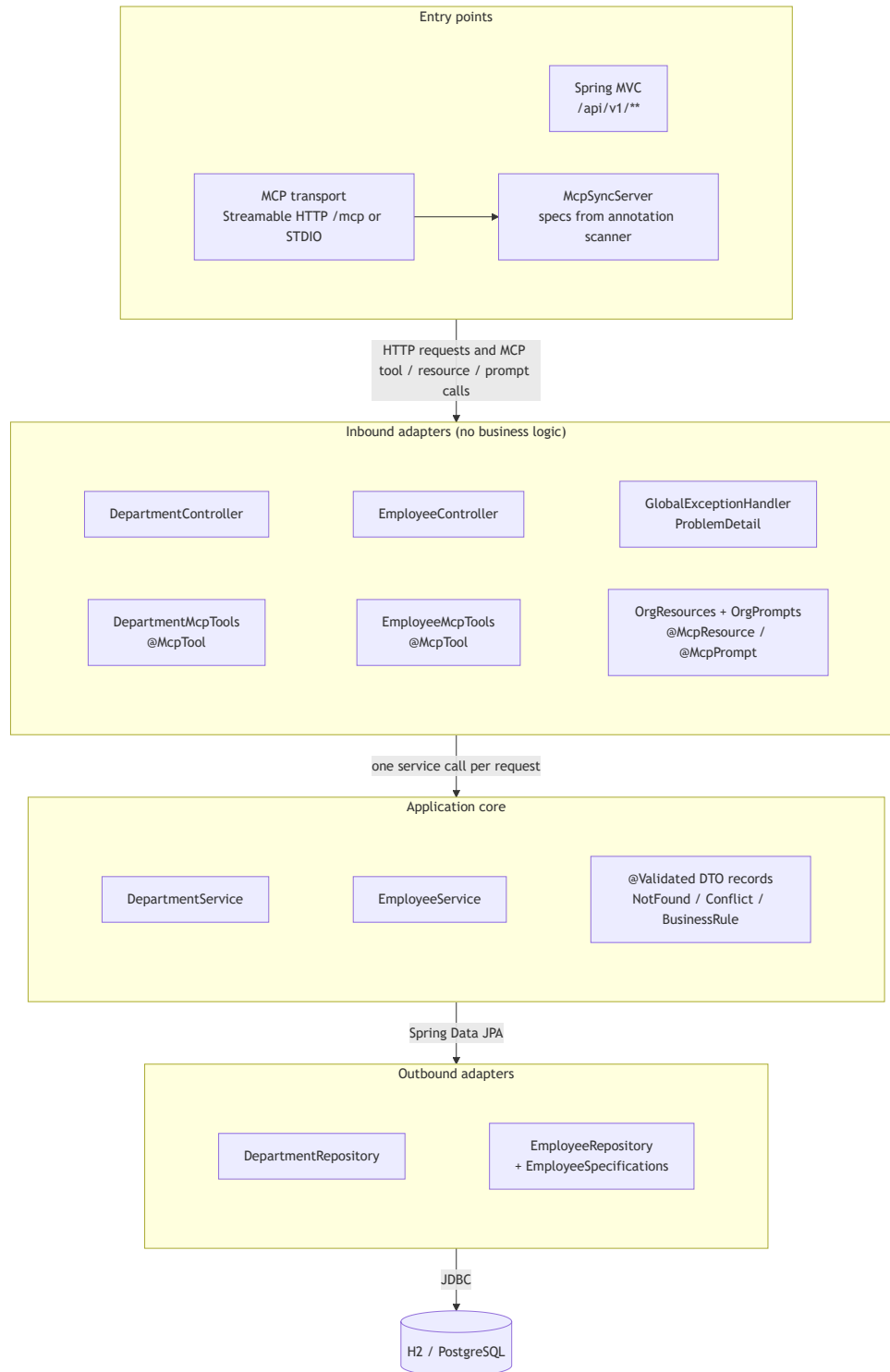
4. Container / deployment view



The same JAR runs in two modes:

Mode	Command	Active interfaces
HTTP (default)	<code>java -jar app.jar</code>	REST, MCP Streamable HTTP, Swagger, Actuator
STDIO	<code>java -jar app.jar --spring.profiles.active=stdio</code>	MCP over stdin/stdout only

5. Component view (hexagonal layering)



Rule: controllers and MCP tool classes contain no business logic. They validate the input shape, call one service method, and return a DTO. This keeps REST and MCP behaviour the same.

5.1 Source layout

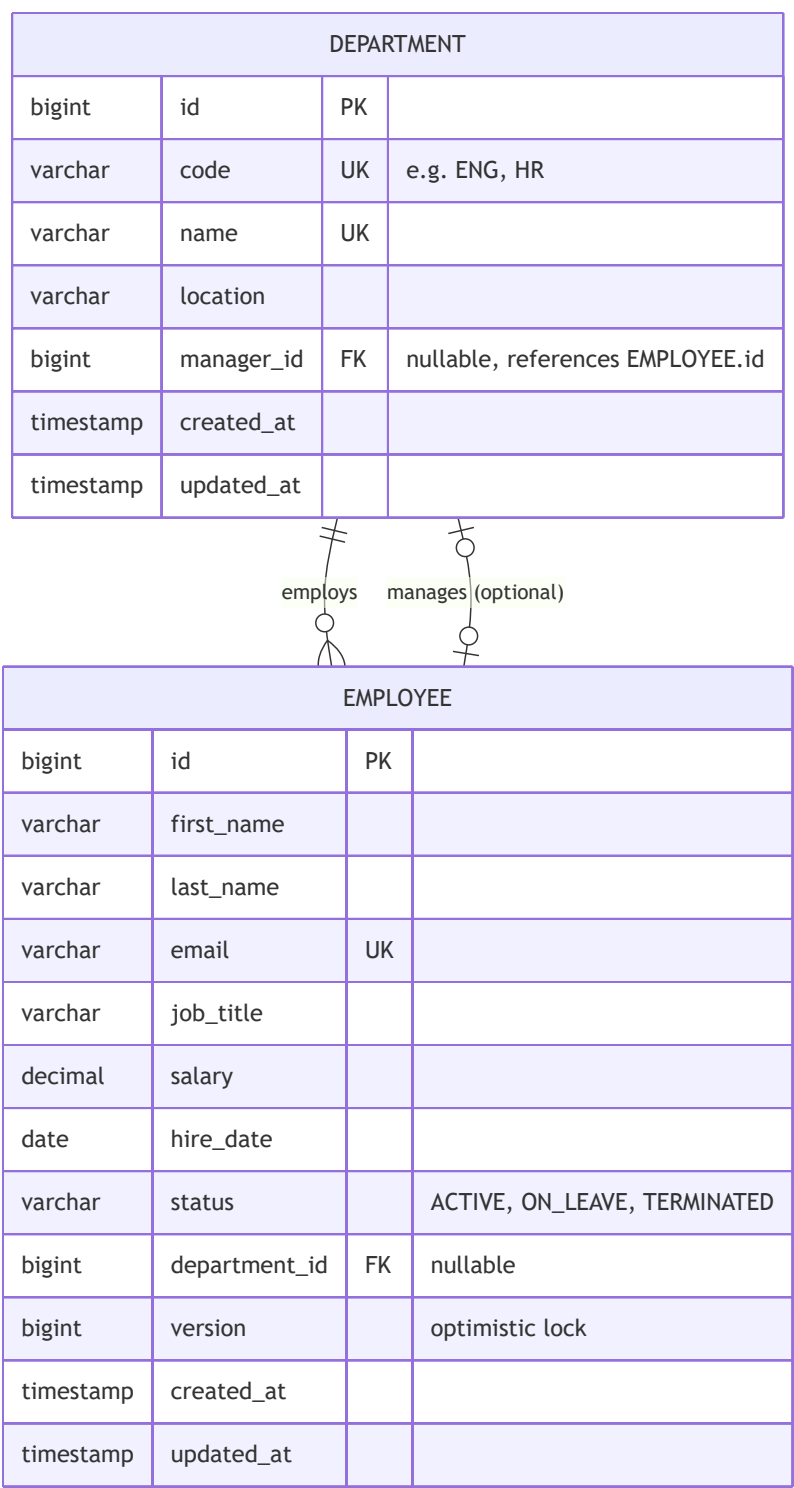
```

src/main/java/com/restmcp/demo/
├─ RestApiMcpDemoApplication.java
├─ common/
│   ├── exception/    NotFoundException, ConflictException, BusinessException, GlobalExceptionHandler
│   └─ api/           PageResponse<T>
├─ department/
│   ├── Department.java          (JPA entity)
│   ├── DepartmentRepository.java
│   ├── DepartmentStats.java, DepartmentHeadcount.java    (query projections)
│   ├── DepartmentService.java
│   ├── DepartmentController.java
│   └─ dto/    DepartmentRequest, DepartmentResponse, DepartmentSummary
├─ employee/
│   ├── Employee.java            (JPA entity)
│   ├── EmployeeStatus.java      (enum)
│   ├── EmployeeRepository.java
│   ├── EmployeeSpecifications.java (dynamic search filter)
│   ├── EmployeeService.java
│   ├── EmployeeController.java
│   └─ dto/    EmployeeRequest, EmployeeResponse, EmployeeSearchCriteria
├─ mcp/
│   ├── DepartmentMcpTools.java
│   ├── EmployeeMcpTools.java
│   ├── OrgResources.java        (@McpResource: directory + roster template)
│   ├── OrgPrompts.java          (@McpPrompt: department_report, onboard_employee)
│   ├── McpToolMetricsAspect.java (timer + log line per tool call)
│   └─ McpOriginValidationFilter.java
├─ config/
│   ├── WebSecurityConfig.java    (registers the filters, web mode only)
│   ├── ApiKeyAuthFilter.java
│   └─ OpenApiConfig.java
src/main/resources/
├─ application.yml
├─ application-stdio.yml
├─ application-legacy-sse.yml
├─ application-postgres.yml
├─ logback-spring.xml            (no console output in the stdio profile)
└─ db/migration/    V1__create_schema.sql, V2__seed_data.sql

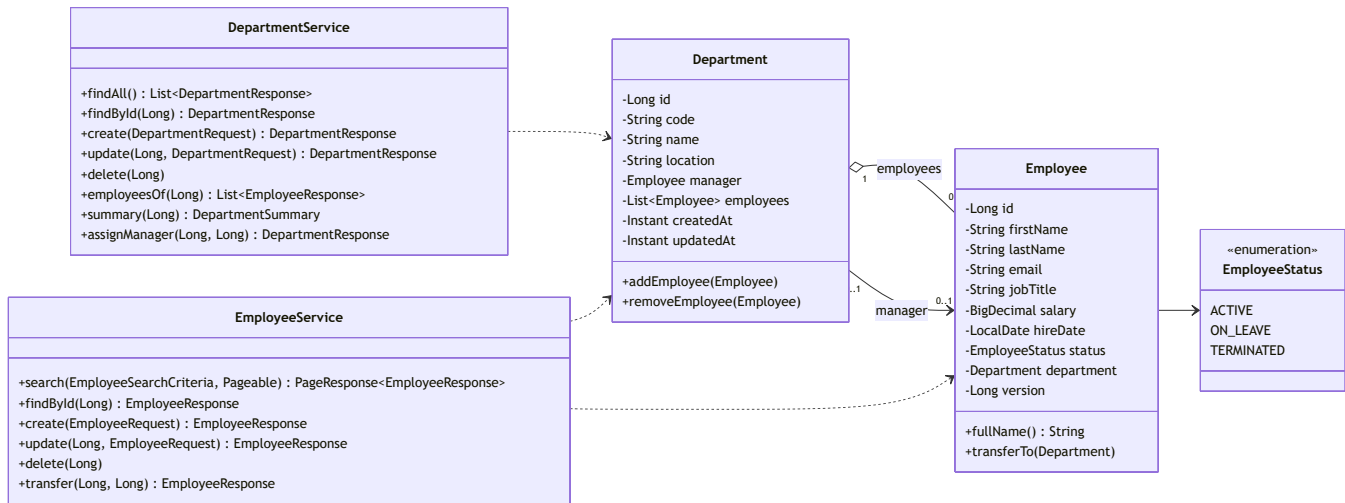
```

6. Domain model

6.1 Entity relationship



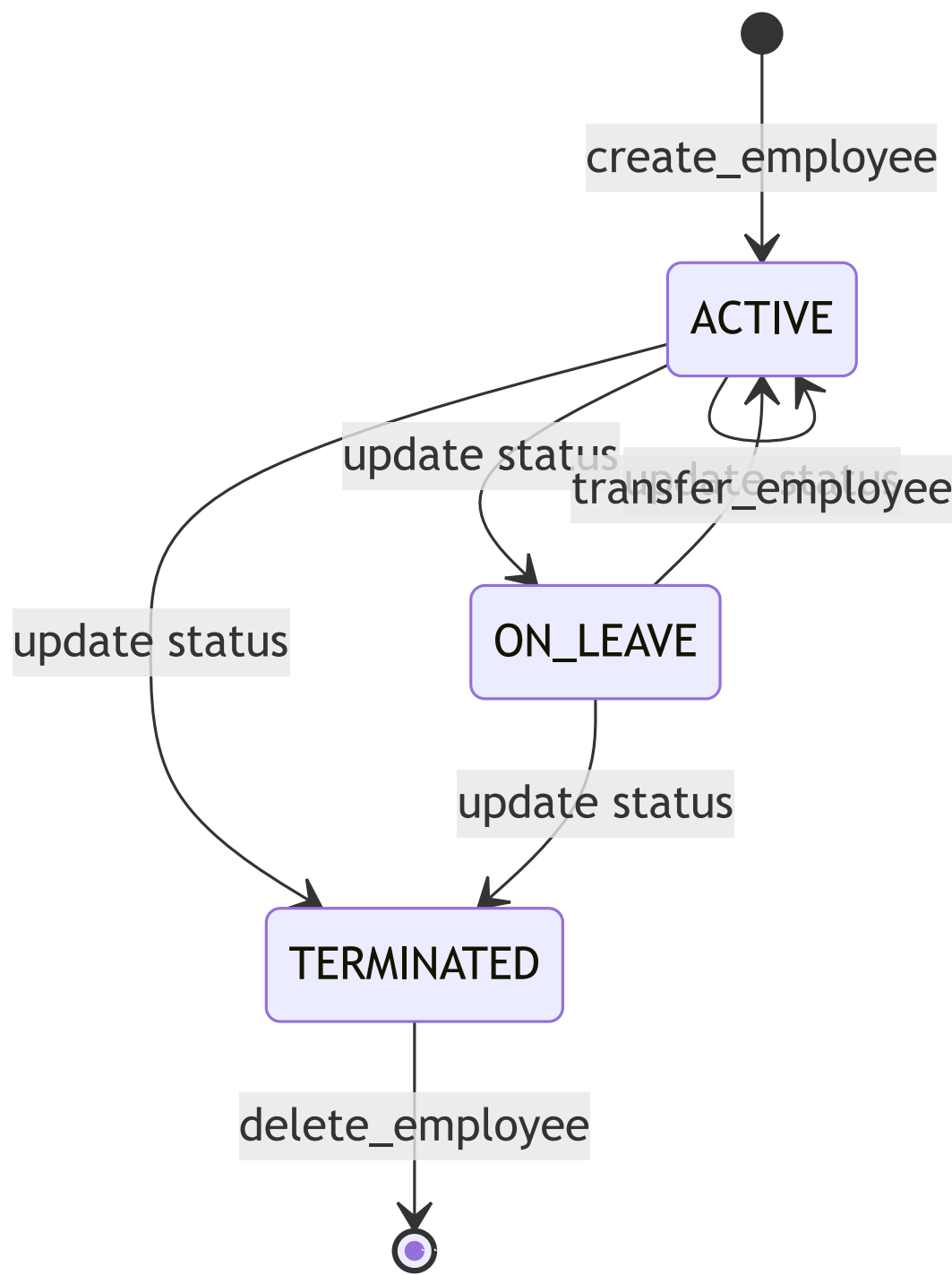
6.2 Class model



6.3 Business rules

Rule	Enforced in	Error
Department <code>code</code> and <code>name</code> are unique	Service + DB unique constraint	409 Conflict / MCP error
Employee <code>email</code> is unique	Service + DB unique constraint	409 Conflict
Salary > 0, names not blank, email well-formed	Bean Validation on request records	400 Bad Request
A department with employees cannot be deleted	<code>DepartmentService.delete</code>	409 Conflict ("reassign N employees first")
A manager must belong to the department they manage	<code>DepartmentService.assignManager</code>	422 Unprocessable
Transferring to the same department is a no-op error	<code>EmployeeService.transfer</code>	422 Unprocessable
A <code>TERMINATED</code> employee cannot be transferred	<code>Employee.transferTo</code>	422 Unprocessable
Concurrent updates	<code>@Version</code> optimistic locking	409 Conflict

6.4 Employee lifecycle



7. REST API design

Base path: `/api/v1` . JSON only. Errors use `application/problem+json` .

7.1 Departments

Method	Path	Description	Success
GET	<code>/departments</code>	List all departments with headcount	200

Method	Path	Description	Success
GET	/departments/{id}	Get one department	200 / 404
POST	/departments	Create a department	201 + Location
PUT	/departments/{id}	Replace a department	200 / 404 / 409
DELETE	/departments/{id}	Delete an empty department	204 / 404 / 409
GET	/departments/{id}/employees	Employees in the department	200 / 404
GET	/departments/{id}/summary	Headcount, total and average salary, manager	200 / 404
PUT	/departments/{id}/manager/{employeeId}	Assign a manager	200 / 404 / 422

7.2 Employees

Method	Path	Description	Success
GET	/employees? name=&departmentId=&status=&page=&size=&sort=	Search with paging	200
GET	/employees/{id}	Get one employee with department info	200 / 404
POST	/employees	Create an employee, optional departmentId	201 / 409
PUT	/employees/{id}	Update an employee	200 / 404 / 409
DELETE	/employees/{id}	Delete an employee	204 / 404
PUT	/employees/{id}/department/{departmentId}	Transfer to another department	200 / 404 / 422

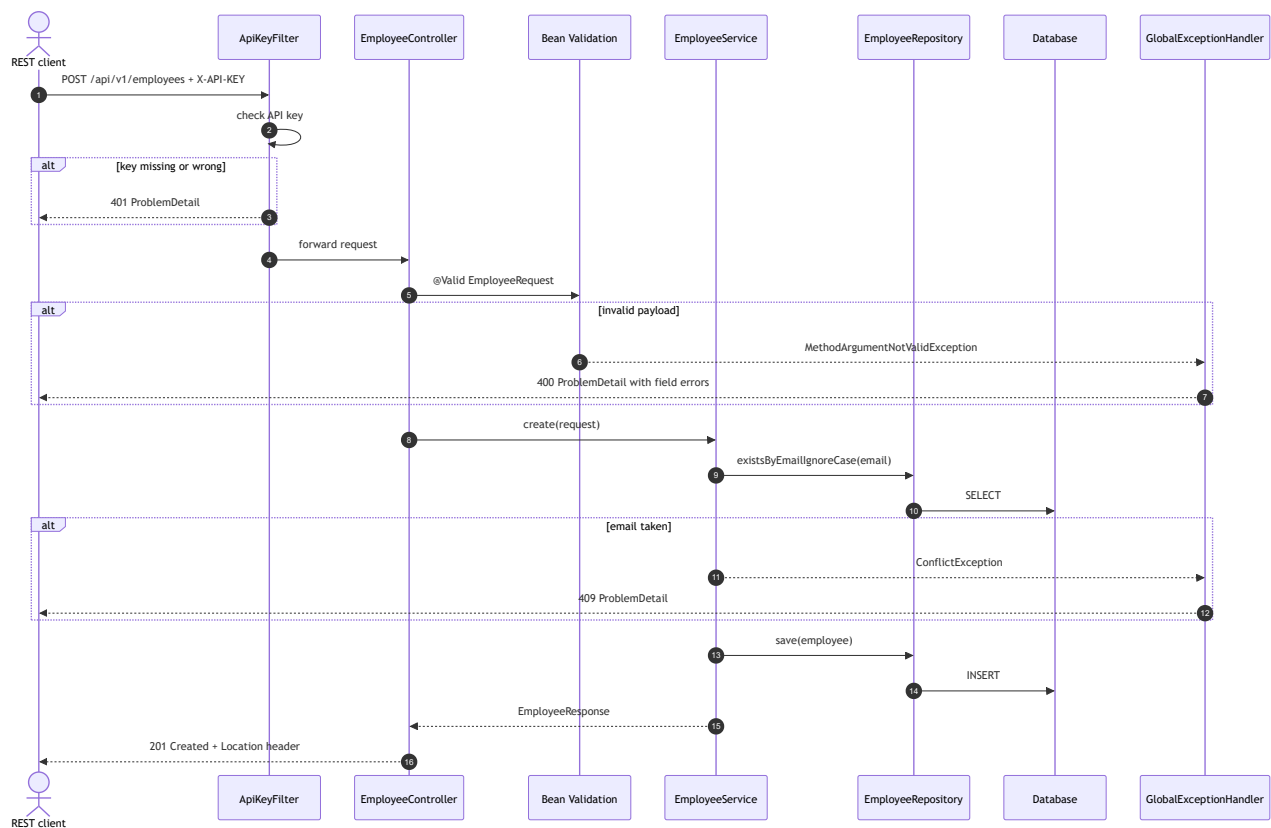
7.3 Payload examples

```
// POST /api/v1/employees
{
  "firstName": "Asha",
  "lastName": "Rao",
  "email": "asha.rao@example.com",
  "jobTitle": "Software Engineer",
  "salary": 95000.00,
  "hireDate": "2026-09-01",
  "departmentId": 1
}
```

```
// 201 Created
{
  "id": 42,
  "firstName": "Asha",
  "lastName": "Rao",
  "fullName": "Asha Rao",
  "email": "asha.rao@example.com",
  "jobTitle": "Software Engineer",
  "salary": 95000.00,
  "hireDate": "2026-09-01",
  "status": "ACTIVE",
  "department": { "id": 1, "code": "ENG", "name": "Engineering" }
}
```

```
// 409 Conflict (application/problem+json)
{
  "type": "https://restmcp.demo/errors/conflict",
  "title": "Conflict",
  "status": 409,
  "detail": "Employee with email 'asha.rao@example.com' already exists",
  "instance": "/api/v1/employees"
}
```

7.4 REST request flow



8. MCP server design

8.1 MCP concepts used

MCP primitive	Controlled by	What we expose
Tools	The model decides when to call them	All read and write operations (§8.2)
Resources	The application or user attaches them as context	<code>org://departments</code> (directory), <code>org://departments/{code}/roster</code> (template)
Prompts	The user picks them, e.g. as slash commands	<code>onboard_employee</code> , <code>department_report</code>

8.2 Tool catalogue

Tool names use `snake_case` . Descriptions are written for an LLM: say what the tool does, when to use it, and what it returns.

Tool	Parameters	Returns	Delegates to	Kind
<code>list_departments</code>	—	<code>DepartmentResponse[]</code>	<code>DepartmentService.findAll</code>	read
<code>get_department</code>	<code>departmentId</code>	<code>DepartmentResponse</code>	<code>findById</code>	read
<code>get_department_employees</code>	<code>departmentId</code>	<code>EmployeeResponse[]</code>	<code>employeesOf</code>	read
<code>get_department_summary</code>	<code>departmentId</code>	<code>DepartmentSummary</code>	<code>summary</code>	read
<code>create_department</code>	<code>code</code> , <code>name</code> , <code>location</code>	<code>DepartmentResponse</code>	<code>create</code>	write
<code>update_department</code>	<code>departmentId</code> , <code>code</code> , <code>name</code> , <code>location</code>	<code>DepartmentResponse</code>	<code>update</code>	write
<code>delete_department</code>	<code>departmentId</code>	confirmation text	<code>delete</code>	destructive
<code>assign_department_manager</code>	<code>departmentId</code> , <code>employeeId</code>	<code>DepartmentResponse</code>	<code>assignManager</code>	write
<code>search_employees</code>	<code>name?</code> , <code>departmentId?</code> , <code>status?</code> , <code>page?</code> , <code>size?</code>	<code>PageResponse<EmployeeResponse></code>	<code>EmployeeService.search</code>	read
<code>get_employee</code>	<code>employeeId</code>	<code>EmployeeResponse</code>	<code>findById</code>	read
<code>create_employee</code>	<code>firstName</code> , <code>lastName</code> , <code>email</code> , <code>jobTitle</code> , <code>salary</code> , <code>hireDate</code> , <code>departmentId?</code>	<code>EmployeeResponse</code>	<code>create</code>	write
<code>update_employee</code>	<code>employeeId</code> + fields	<code>EmployeeResponse</code>	<code>update</code>	write
<code>delete_employee</code>	<code>employeeId</code>	confirmation text	<code>delete</code>	destructive
<code>transfer_employee</code>	<code>employeeId</code> , <code>targetDepartmentId</code>	<code>EmployeeResponse</code>	<code>transfer</code>	write

Write and destructive tools say so in their description ("Permanently deletes..."), so MCP hosts that ask the user to confirm tool calls show a clear message.

8.3 Tool implementation pattern

```
@Component
public class EmployeeMcpTools {

    private final EmployeeService employeeService;

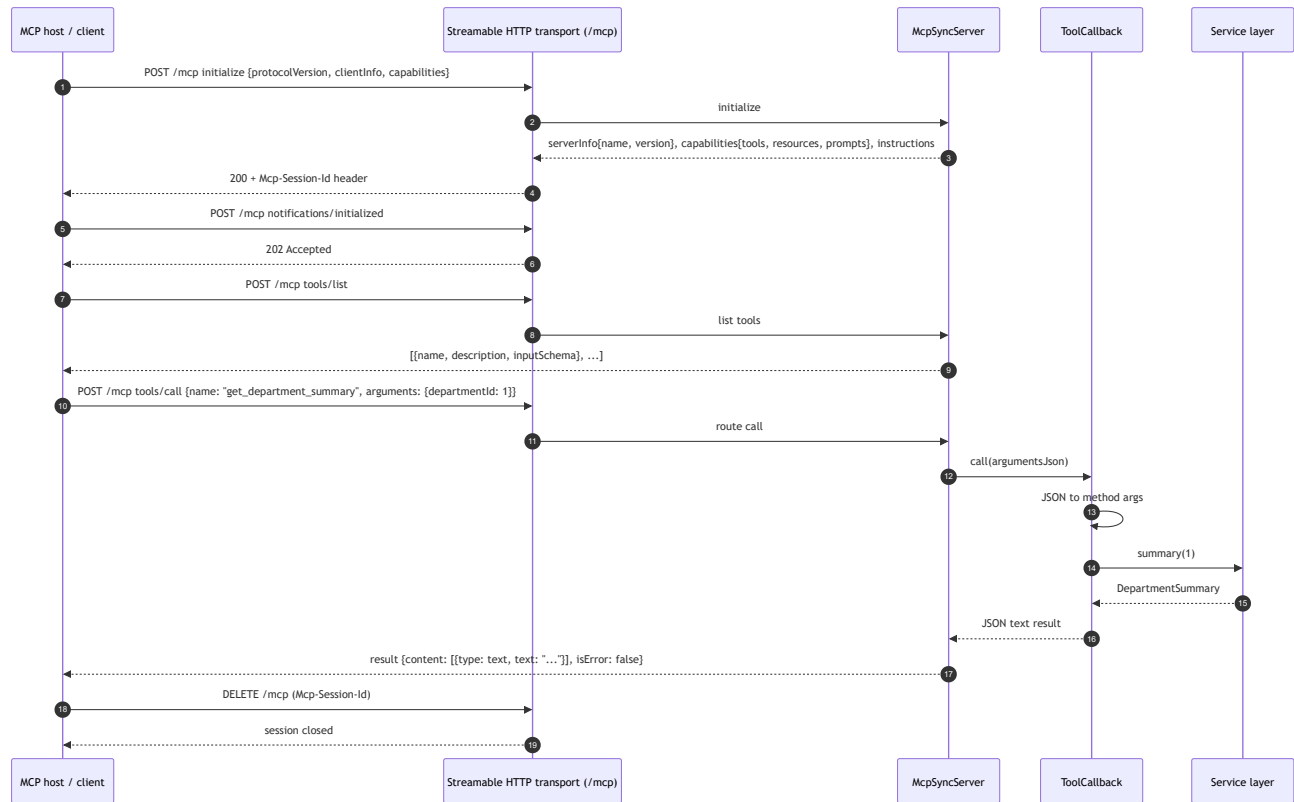
    public EmployeeMcpTools(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    @McpTool(name = "transfer_employee",
        description = "Move an employee to a different department. Find the employee id with "
            + "search_employees and the department id with list_departments first. TERMINATED employees "
            + "cannot be moved. Returns the updated employee.",
        annotations = @McpAnnotations(readOnlyHint = false, destructiveHint = false, idempotentHint = false,
            openWorldHint = false))
    public EmployeeResponse transferEmployee(
        @McpToolParam(description = "Numeric id of the employee to move") Long employeeId,
        @McpToolParam(description = "Numeric id of the destination department") Long targetDepartmentId) {
        return employeeService.transfer(employeeId, targetDepartmentId);
    }
}
```

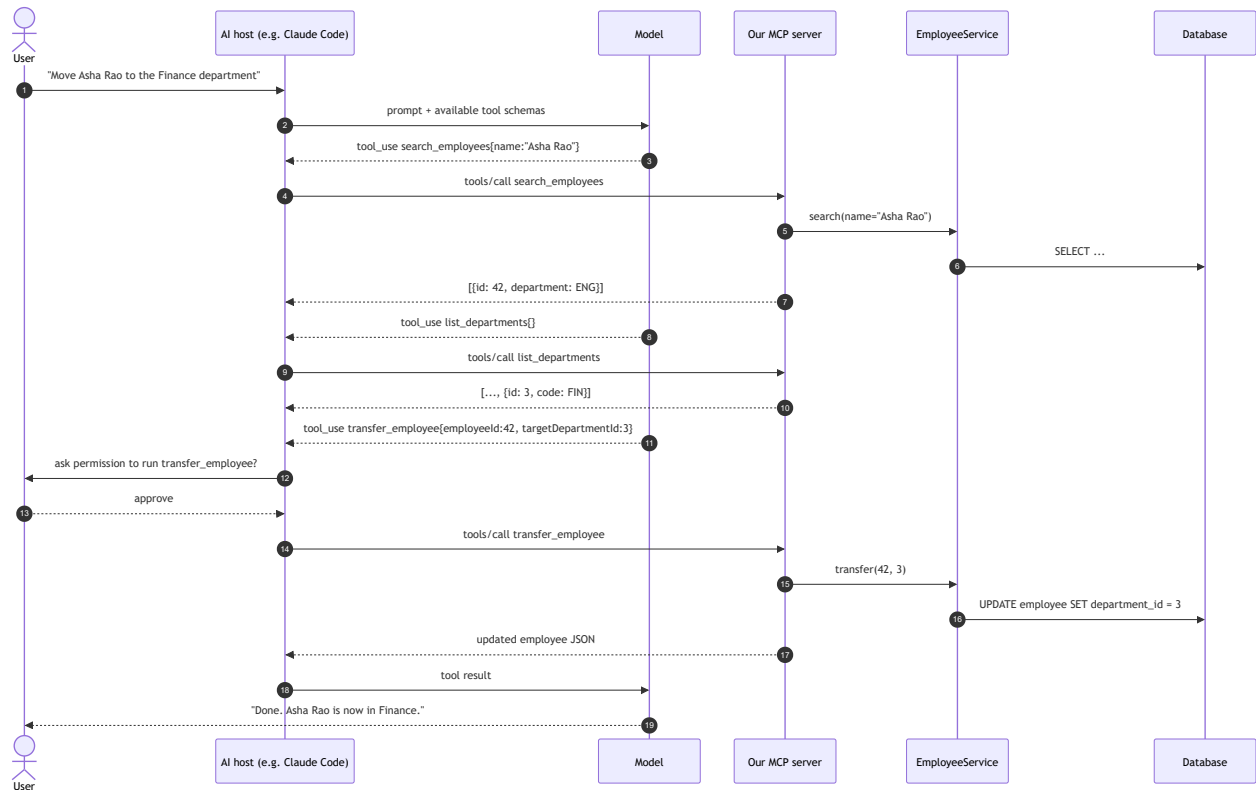
The Spring AI annotation scanner finds `@McpTool` methods on Spring beans, builds each tool's JSON input schema from the method signature and `@McpToolParam` descriptions (`LocalDate` becomes `"format": "date"` , enums become `enum` lists), and registers them with the `McpSyncServer` . No registration bean is needed. Update tools take optional fields and merge them with the current record, so the model only sends what changes.

Service methods are `@Validated` with `@Valid` request records, so MCP calls get the same Bean Validation as REST requests. A snapshot of every tool's name, description, hints and schema is kept in `src/test/resources/mcp/tools-snapshot.json` , and a test fails if it changes unexpectedly.

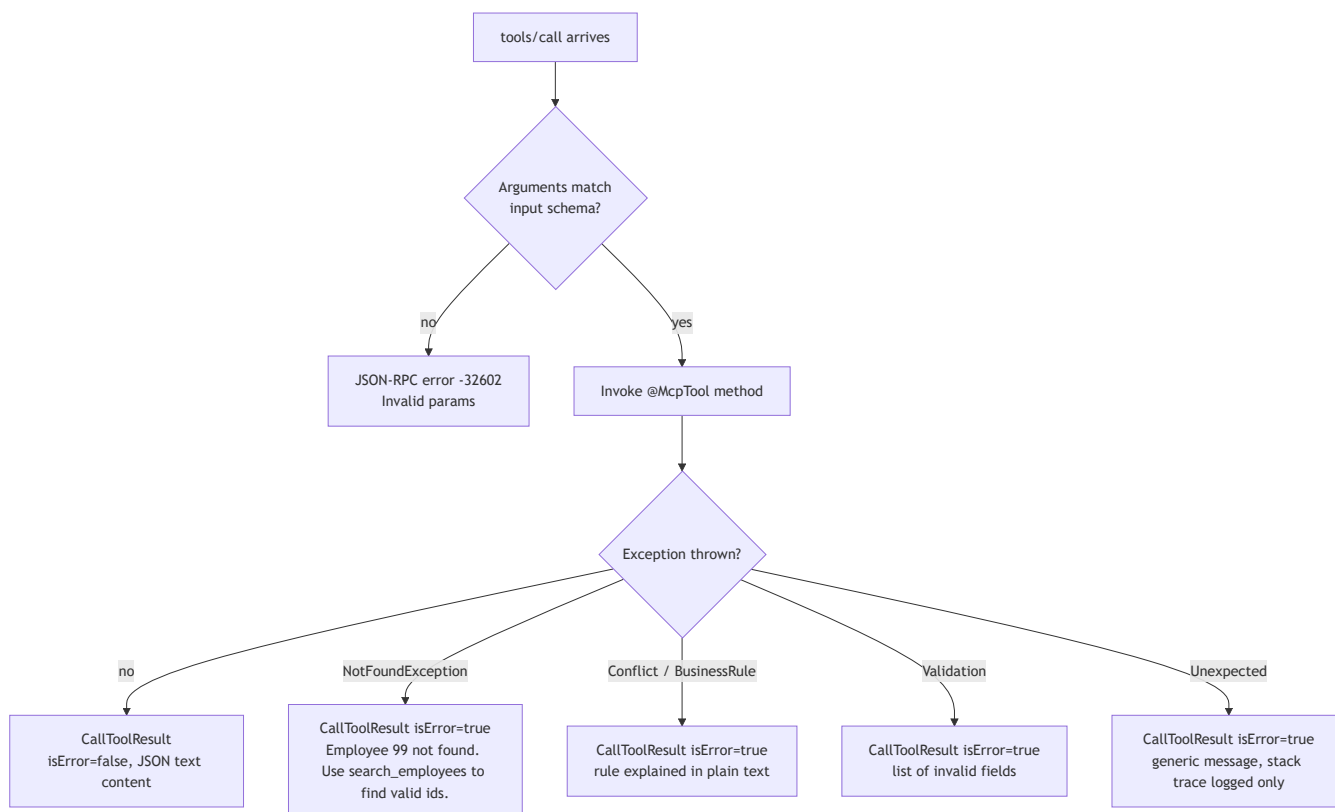
8.4 MCP session lifecycle (Streamable HTTP)



8.5 End-to-end: a user prompt becomes a tool call

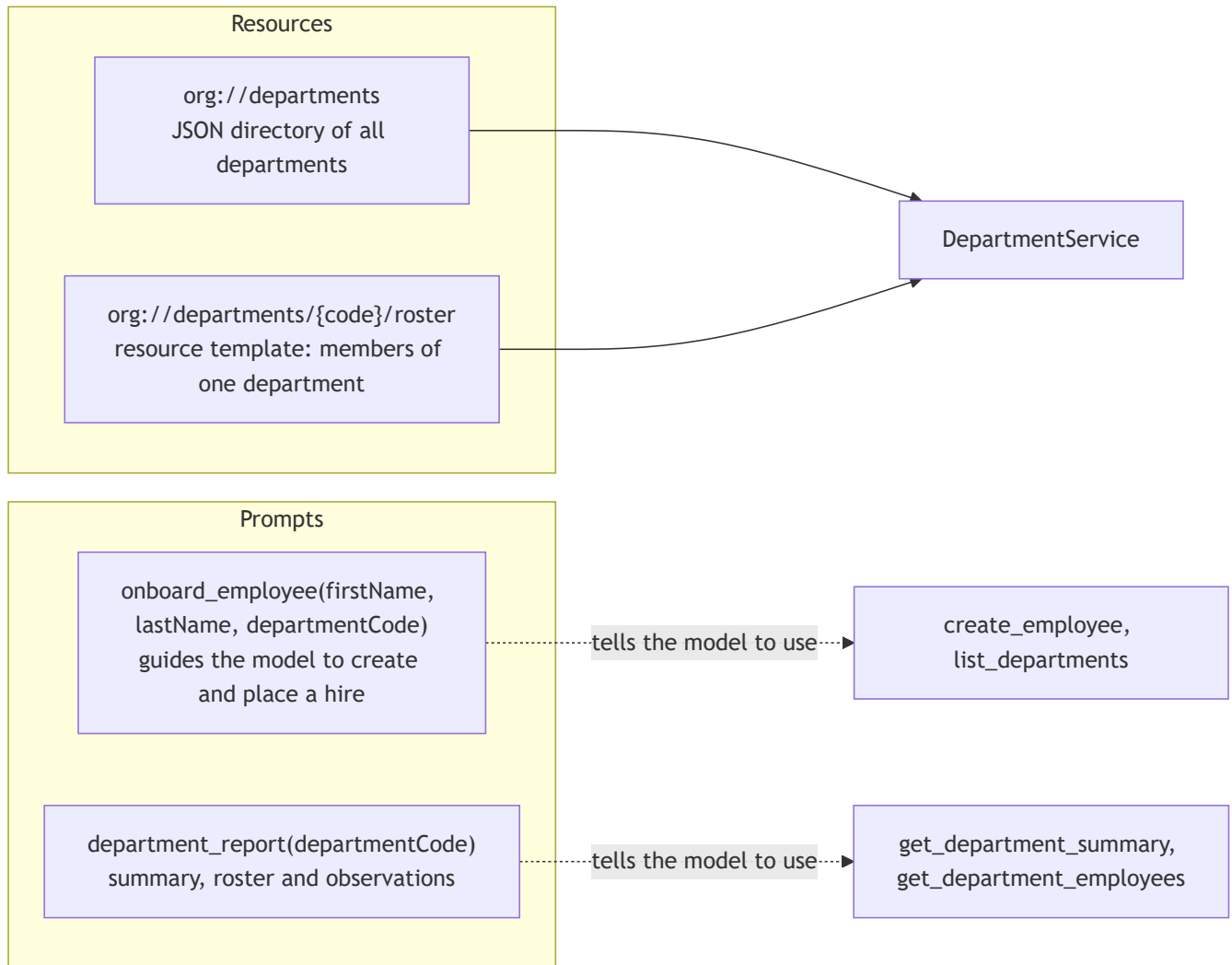


8.6 MCP error handling



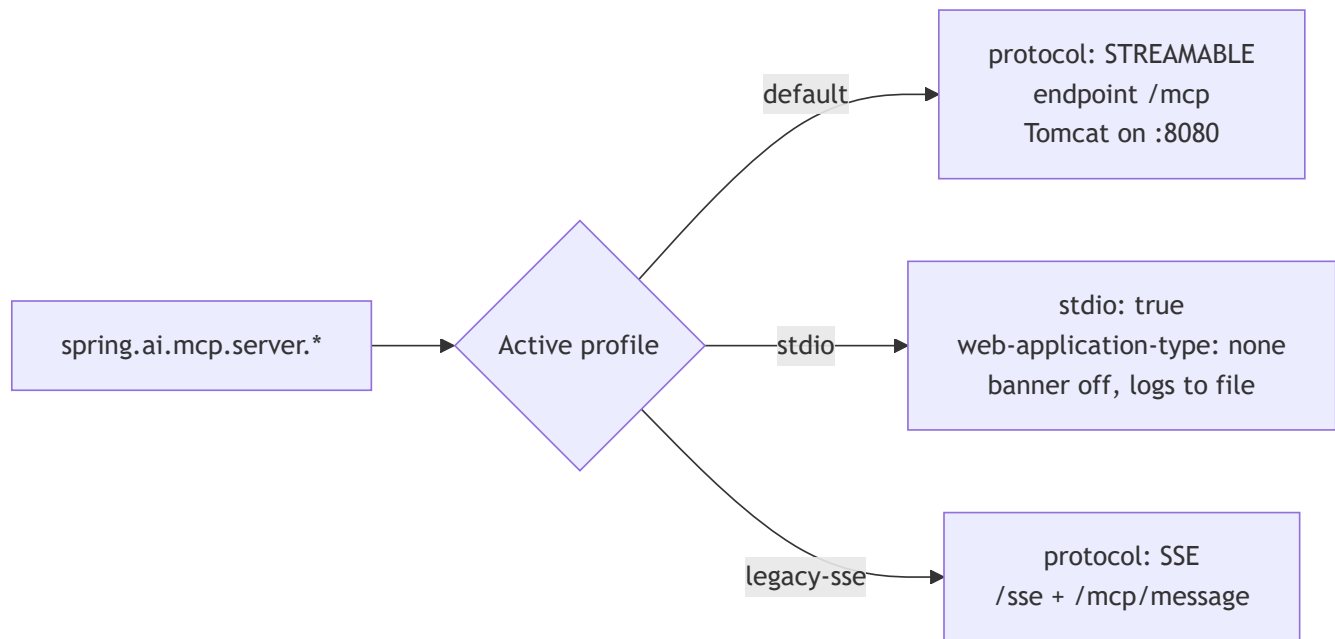
Business errors come back as **tool results with `isError=true`**, not as protocol errors. The model can read the message and try again, for example by looking up a valid id. Domain exception messages are written to be useful to the model. Internal details such as SQL or stack traces are never returned.

8.7 Resources and prompts



Implemented with `@McpResource ( OrgResources )` and `@McpPrompt / @McpArg ( OrgPrompts )`. The roster URI contains a `{code}` variable, so it is published as a resource *template* (`resources/templates/list`).

8.8 Transport configuration



`application.yml` (default, HTTP):

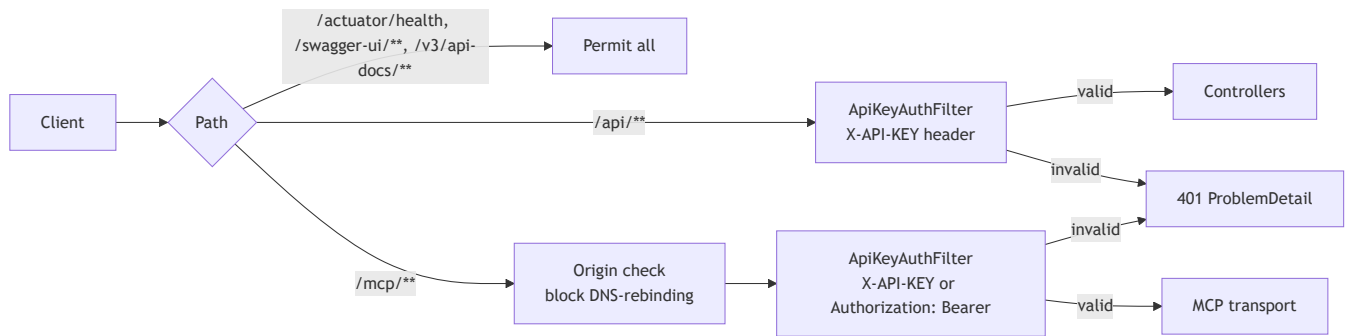
```

spring:
  application:
    name: rest-api-mcp-demo
  ai:
    mcp:
      server:
        name: employee-department-mcp
        version: 1.0.0
        type: SYNC
        protocol: STREAMABLE
        instructions: >
          Tools for an organisation's employees and departments. Look up ids with
          list_departments or search_employees before calling tools that need an id.
          Confirm with the user before delete_* or transfer_employee.
        capabilities:
          tool: true
          resource: true
          prompt: true
          completion: false
        streamable-http:
          mcp-endpoint: /mcp
  
```

`application-stdio.yml`:

```
spring:
  main:
    web-application-type: none
    banner-mode: off
  ai:
    mcp:
      server:
        stdio: true
  logging:
    pattern:
      console:          # nothing may be written to stdout except JSON-RPC
    file:
      name: ${java.io.tmpdir}/rest-api-mcp-demo.log
```

9. Security architecture



Concern	Control
Authentication	API key from <code>APP_API_KEY</code> env var, compared in constant time. Off in the <code>dev</code> profile for easy demos.
DNS rebinding (MCP spec requirement for HTTP servers)	Validate the <code>Origin</code> header on <code>/mcp</code> . Bind to <code>127.0.0.1</code> by default (<code>server.address</code>).
Destructive operations	Tool descriptions mark them clearly. Hosts ask the user before calling.
Data exposure	DTOs only. Salary can be dropped from responses with a feature flag if needed.
Secrets	Never in <code>application.yml</code> . Read from env vars or a secret store.
Upgrade path	<code>spring-boot-starter-oauth2-resource-server</code> + JWT validation on <code>/mcp</code> , publish OAuth Protected Resource Metadata (<code>/well-known/oauth-protected-resource</code>) as the MCP authorization spec requires.

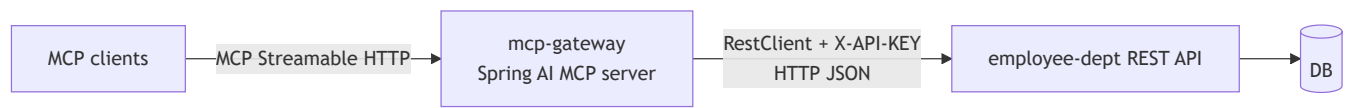
10. Cross-cutting concerns

Concern	Approach
Validation	Jakarta Bean Validation on request records (<code>@NotBlank</code> , <code>@Email</code> , <code>@Positive</code> , <code>@PastOrPresent</code>). The service re-checks uniqueness rules.
Transactions	<code>@Transactional(readOnly = true)</code> on the service class, <code>@Transactional</code> on write methods.
N+1 queries	<code>@EntityGraph(attributePaths = "department")</code> on employee queries. Headcount via a <code>GROUP BY</code> projection.
Paging	<code>Pageable</code> with a max page size of 100, returned as <code>PageResponse<T>{content, page, size, totalElements, totalPages}</code> so MCP JSON stays stable.

Concern	Approach
Observability	Actuator health/info/metrics. Log line per tool call (<code>tool</code> , duration, outcome). Spring AI MCP observations through Micrometer.
API docs	springdoc-openapi at <code>/swagger-ui.html</code> . The MCP tool list is visible through MCP Inspector.
Config	Profiles: <code>default</code> (H2, HTTP), <code>stdio</code> , <code>postgres</code> , <code>dev</code> (security off).
Packaging	Executable JAR plus a Docker image (<code>eclipse-temurin:21-jre</code> , non-root user).

11. Alternative topology: MCP gateway in front of an existing REST API

Use this when the REST service already exists, is owned by another team, or is written in another language.

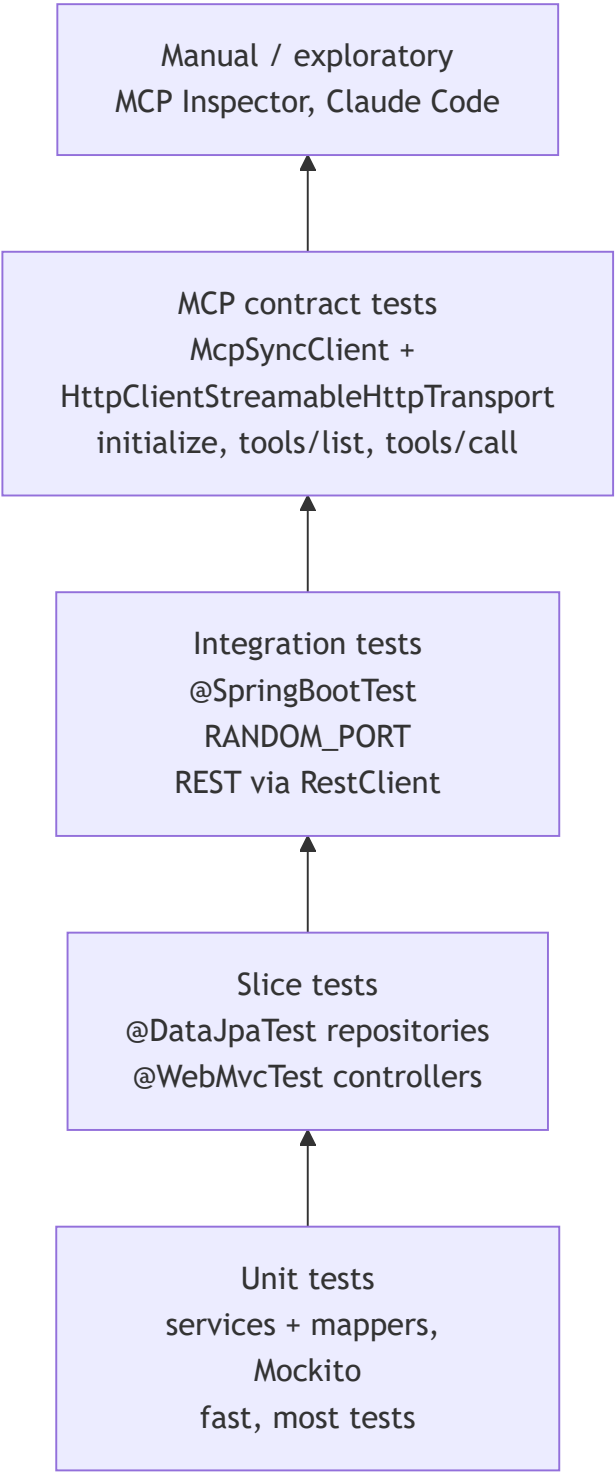


The `@McpTool` classes stay the same, but they call a `RestClient`-based `EmployeeApiClient` instead of the service. The gateway must translate HTTP 4xx `ProblemDetail` responses into `isError` tool results. Cost: an extra network hop, two deployables and two error mappings. This repo uses the in-process design (ADR-01). The plan structures the code so the gateway can be split out later without rewriting the tools.

12. Client integration matrix

Client	Transport	Configuration
Claude Code	HTTP	<code>claude mcp add --transport http emp-dept http://localhost:8080/mcp --header "X-API-KEY: <key>"</code>
Claude Desktop	STDIO	<code>claude_desktop_config.json : { "mcpServers": { "emp-dept": { "command": "java", "args": ["-jar", "<abs path>/rest-api-mcp-demo.jar", "--spring.profiles.active=stdio"] } } }</code>
Claude Desktop (remote)	HTTP via bridge	<code>{ "command": "npx", "args": ["-y", "mcp-remote", "http://localhost:8080/mcp", "--header", "X-API-KEY: <key>"] }</code>
VS Code (Copilot agent mode)	HTTP	<code>.vscode/mcp.json : { "servers": { "emp-dept": { "type": "http", "url": "http://localhost:8080/mcp", "headers": { "X-API-KEY": "<key>" } } } }</code>
Cursor	HTTP	<code>~/.cursor/mcp.json : { "mcpServers": { "emp-dept": { "url": "http://localhost:8080/mcp" } } }</code>
MCP Inspector	HTTP or STDIO	<code>npx @modelcontextprotocol/inspector</code> , then choose "Streamable HTTP" and <code>http://localhost:8080/mcp</code>
Spring AI client	HTTP	<code>spring.ai.mcp.client.streamable-http.connections.emp-dept.url=http://localhost:8080</code>

13. Testing strategy



Level	Proves
Unit	Business rules (§6.3) and mapping
@DataJpaTest	Flyway schema, constraints, custom queries, entity graphs
@WebMvcTest	Status codes, validation, ProblemDetail shape, JSON contract
REST integration	Whole stack with the real DB and security filter

Level	Proves
MCP contract	The server advertises exactly the expected tool names and schemas; each tool returns correct data; business errors come back as <code>isError=true</code>
STDIO smoke	The JAR in the <code>stdio</code> profile writes only JSON-RPC to stdout

14. Glossary

Term	Meaning
MCP	Model Context Protocol: an open JSON-RPC 2.0 protocol for connecting AI hosts to tools and data
Host	The AI application (Claude Desktop, IDE) that runs MCP clients
Tool	A callable function with a JSON input schema, invoked when the model decides to
Resource	Read-only context addressed by URI
Prompt	A reusable, parameterised prompt template the user picks
Streamable HTTP	MCP transport over a single HTTP endpoint (POST for requests, optional SSE stream for responses)
STDIO	MCP transport where the host launches the server and talks over stdin/stdout

15. Implementation notes: where the build differs from the draft

Area	Draft assumption	As built	Why
Versions	Boot 3.5 + Spring AI 1.1	Boot 4.1.1 + Spring AI 2.0.1 (MCP SDK 2.0, Jackson 3, Hibernate 7)	start.spring.io no longer offers Boot 3.5. Boot 4 moved test annotations (<code>org.springframework.boot.webmvc.test...</code> , <code>data.jpa.test...</code>) and <code>PropertyReferenceException</code> (<code>org.springframework.data.core</code>).
Tool API	<code>@Tool</code> + <code>ToolCallbackProvider</code> bean	<code>@McpTool</code> / <code>@McpToolParam</code> scanned automatically	Publishes MCP tool annotations (read-only / destructive hints).
Transport default	<code>protocol</code> defaults to Streamable HTTP	Must set <code>spring.ai.mcp.server.protocol: STREAMABLE</code> explicitly	Without it the auto-configuration activates the legacy SSE transport and <code>/mcp</code> returns 404 (found during Phase 3 smoke testing).
Validation	Controllers only	Also <code>@Validated</code> services	An MCP contract test showed invalid emails were accepted through tools.
Search	JPQL with nullable parameters	JPA Specification (<code>EmployeeSpecifications</code>)	Avoids null-parameter type inference problems on PostgreSQL.
Manager mapping	<code>@OneToOne</code>	<code>@ManyToOne</code> on <code>Department.manager</code>	Simpler lazy loading. Each department still has at most one manager.
Timestamps	<code>TIMESTAMP</code>	<code>TIMESTAMP WITH TIME ZONE</code>	<code>Instant</code> maps to <code>timestamptz</code> in Hibernate 7, so <code>ddl-auto: validate</code> passes on PostgreSQL.
Security	Key always required	Required only when <code>APP_API_KEY</code> is set	Zero-setup local demo, still bound to 127.0.0.1. Compose refuses to start without a key.
Observability	Metrics as an idea	<code>mcp.tool.calls</code> timer (tags <code>tool</code> , <code>outcome</code>) via <code>McpToolMetricsAspect</code> , plus one log line per call	

Area	Draft assumption	As built	Why
Verification	Unit + integration tests	Also run end-to-end: container image on PostgreSQL 16 via Podman (migrations, <code>ddl-auto: validate</code> , search, transfer, 401 without key), and Claude Code calling the tools through MCP (read, write, business-error paths)	Docker Desktop not required; Podman with compose works as-is.
STDIO state	Not considered	In-memory H2 means each STDIO launch has its own data	Use the <code>postgres</code> profile to share state between transports.